

南京工程学院

信息与通信工程学院

实 验 报 告

课 程 名 称 嵌入式系统编程实训

实验项目名称 文件操作

实验学生班级 信息17

实验学生姓名 余卓豫

同组学生姓名 208170809

实 验 时 间 6月20日

实 验 地 点 _____

实验成绩评定 _____

指导教师签字 _____

年 月 日

一. 实验目的

掌握Linux文件操作

二. 实验设备

- 1台PC机、嵌入式教学实验开发平台
- Fedora Linux操作系统、嵌入式Linux开发环境

三. 实验主要内容

1. 应用程序编程、编译、链接、运行、调试的过程, 针对目录和文件的编程操作
2. 编写hello world、who、cp、tree、ls.

四. 实验步骤

1. hello.c

(1). 代码:

```
#include <stdio.h>
main() {
    printf("hello world!");
}
```

(2). 结果如下:

```
./hello
hello world
```

2. cp.c

(1). 代码:

```
#define BUFFER_SIZE 4096 // 该取字符串长度
#define COPY_MODE 0644 // 文件权限
void opps(char *x, char *y); // 函数声明
main(int ac, char *av[]) { // 命令行模式
    int in_fd, out_fd, n_chars;
    char buf[BUFFER_SIZE];
    if (ac != 3) { // 当命令行参数不为3时, 输出错误信息
        fprintf(stderr, "usage: %s source destination\n", *av); // 输出 av[0]
        exit(1); // 异常退出
    }
    if ((in_fd = open(av[1], O_RDONLY)) == -1) { // 打开源文件失败
        opps("open error", av[1]); // 输出错误信息
    }
    if ((out_fd = creat(av[2], COPY_MODE)) == -1) { // 创建文件失败
        opps("create error", av[2]); // 输出错误信息
    }
    while ((n_chars = read(in_fd, buf, BUFFER_SIZE)) > 0) { // 该取成功时
        if (write(out_fd, buf, n_chars) != n_chars) { // 该取长度不等, 则为失败
            opps("write error", av[2]); // 输出错误信息
        }
        if (n_chars == 1)
            opps("Read error from", av[1]);
    }
}
```

```

if (close (in-fd) == -1 || close (out-fd) == -1)
    opps (" Error closing files ", " "); }
void opps (char *s1, char *s2) { // 输出错误信息.
    fprintf (stderr, "Error=%s", s1);
    perror (s2);
    exit (1);
}

```

12) 代码运行如下.

```

[root@cvtech shiyan1]# ./hello
hello world[root@cvtech shiyan1]# ./cp
usage: ./cp source destination

```

3. who.c

1). 代码:

```

int show_info (struct utmp * utbuf);
void show_time (time_t);
int main () {
    int fd = open (UTMP_FILE, O_RDONLY); // 定义并初始化文件描述符.
    if (fd == -1) {
        perror (UTMP_FILE); // 打开文件失败, 输出错误信息.
        exit (1);
    }
    struct utmp cur_record; // 读取信息格式.
    int len = sizeof (cur_record);
    while (read (fd, &cur_record, len) == len) {
        show_info (&cur_record); // 读取文件信息, 写出信息.
    }
    close (fd);
    return 0;
}
int show_info (struct utmp * utbuf) {
    printf (" %-8s ", utbuf->ut_user); // 格式化输入信息.
    printf (" ");
    printf (" %-8s ", utbuf->ut_line);
    printf (" ");
    char *cp = ctime (&(utbuf->ut_tv).tv_sec);
    time_t time = time();
    printf ("\n");
    return 0;
}
void show_time (time_t tm) {
    char *cp;
    cp = ctime (&tm); printf (" %20.20s ", cp); // 格式化时间
}

```

12. 运行结果

```
[root@cvtech shiyan1]# ./who
reboot      -      Sun Jun 21 10:54:23
runlevel    -      Sun Jun 21 10:54:23
LOGIN       tty4      Sun Jun 21 10:54:27
LOGIN       tty5      Sun Jun 21 10:54:27
LOGIN       tty2      Sun Jun 21 10:54:27
LOGIN       tty3      Sun Jun 21 10:54:27
LOGIN       tty6      Sun Jun 21 10:54:27
root        :0      Sun Jun 21 10:54:49
root        pts/0    Sun Jun 21 18:48:43
root        pts/1    Tue Jun 23 21:12:20
```

4. ls.c

1) 代码如下.

```
char get_mode(mode_t m, char *str){
    if(S_ISREG(m)) strcpy(str, "-");
    else if(S_ISDIR(m)) strcpy(str, "d"); // 文件类型为d - 目录.
    else if(S_ISCHR(m)) strcpy(str, "c"); // 文件类型为c - 字符设备.
    else if(S_ISBLK(m)) strcpy(str, "b"); // 文件类型为b - 块设备.
    else if(S_ISFIFO(m)) strcpy(str, "f");
    else if(S_ISLNK(m)) strcpy(str, "l");
    else if(S_ISSOCK(m)) strcpy(str, "n");

    strcat(str, m & S_IRUSR? "r": "-");
    strcat(str, m & S_IWUSR? "w": "-");
    strcat(str, m & S_IXUSR? "x": "-");

    strcat(str, m & S_IRGRP? "r": "-");
    strcat(str, m & S_IWGRP? "w": "-");
    strcat(str, m & S_IXGRP? "x": "-");

    strcat(str, m & S_IROTH? "r": "-");
    strcat(str, m & S_IWOTH? "w": "-");
    strcat(str, m & S_IXOTH? "x": "-");

    return str; }

int _time(int year){ // 设置日期
    if(year % 4 == 0 && year % 100 != 0 || year % 400 == 0)
        return 29;
    return 28; }

wid time_ch(time_t num){ // 设置日期
    int year = 1970;
    int month = 1;
    int day = 1;
    num = num + 8 * 3600;
    while(num >= 86400){
        num -= 86400;
        day++;
        if(month == 1 && day == 32){
```

```

    month++;
    day=1; }
else if (month == 2 && day == -time (year)+1) {
    month++;
    day=1; }
else if (month == 3 && day == 32) {
    month++;
    day=1; }
else if (month == 4 && day == 31) {
    month++;
    day=1; }
else if (month == 5 && day == 32) {
    month++;
    day=1; }
else if (month == 6 && day == 31) {
    month++;
    day=1; }
else if (month == 7 && day == 32) {
    month++;
    day=1; }
else if (month == 8 && day == 32) {
    month++;
    day=1; }
else if (month == 9 && day == 31) {
    month++;
    day=1; }
else if (month == 10 && day == 32) {
    month++;
    day=1; }
else if (month == 11 && day == 31) {
    month++;
    day=1; }
else if (month == 12 && day == 32) {
    month=1;
    day=1;
    year++; }
}

int hour = num/3600;
int minute = num/60 - hour*60;
printf ("%2d月 %2d %2d : %2d ", month, day, hour, minute); ]
int main (int argc, char ** argv, char ** environ) {
    char * dir_name = NULL;
    if (argc == 1) {
        dir_name = "."; }
    char * dir_name = NULL;

```

```

if (argc == 1) {
    dir_name = ".";
} else if (argc == 2) {
    dir_name = argv[1];
} else {
    puts("user: ls dir");
    return -1;
}

DIR *dp = opendir(dir_name);
if (NULL == dp) {
    perror("opendir");
    return -1;
}

struct dirent *de = readdir(dp);
for (; de = readdir(dp);) {
    if ('.' == de->d_name[0]) continue;
    struct stat s;
    int ret = lstat(de->d_name, &s);
    if (0 > ret) {
        perror("stat");
        return -1;
    }

    char str[11] = {};
    printf("%s", get_mode(s.st_mode, str)); // 格式化输出
    struct passwd *passwd;
    passwd = getpwuid(s.st_uid);           // 用户名
    printf("%s", group->gr_name);         // 用户组
    printf("%5lu", s.st_size);            // 文件大小
    time_ch(s.st_mtime);                  // 访问时间
    printf("%s\t", de->d_name);
    printf("\n");
    closedir(dp);
}

```

(2). 运行结果.

```

[root@cvtech shivan1]# ./ls
-rwxrwxr-x root root 6612 6月 21 10:13 tree
-rwxrwxr-x root root 6067 6月 21 10:12 cp
-rw-r--r-- root root 777 6月 21 10:11 cp.c
-rwxrwxr-x root root 5826 6月 21 10: 8 who
-rw-r--r-- root root 54 6月 21 10: 3 hello.c
-rwxrwxr-x root root 8162 6月 21 10:10 ls
-rw-r--r-- root root 1713 6月 21 10:12 tree.c
-rw-r--r-- root root 3149 6月 21 10:10 ls.c
-rwxrwxr-x root root 4857 6月 21 10: 3 hello
-rw-r--r-- root root 929 6月 21 10: 8 who.c
[root@cvtech shivan1]#

```

5. tree.c

(1). 代码.

```

#define MAXNAME200
void s_printf(char *filename, int depth);

```

```

void s_dirwalk(char *dirname, int depth, void (*fcn)(char *, int));
void listdirtree(char *dirname, int depth);

int main(int argc, char **argv) { // 命令的参数
    if (argc == 1)
        listdirtree(".", 0);
    else {
        printf("%s\n", argv[1]); // 打印
        listdirtree(argv[1], 0);
        return 0;
    }
}

void listdirtree(char *dirname, int depth) {
    struct stat stbuf;
    if ((stat(dirname, &stbuf)) == -1) {
        fprintf(stderr, "listdirtree: can't read file %s information!\n", dirname);
        return;
    }
    if ((stbuf.st_mode & S_IFMT) == S_IFDIR)
        s_dirwalk(dirname, depth, listdirtree);
}

void s_dirwalk(char *dirname, int depth, void (*fcn)(char *, int)) {
    char name[MAXNAME];
    struct dirent *fip;
    DIR *dp;
    if ((dp = opendir(dirname)) == NULL) { // 如果打不开目录失败
        fprintf(stderr, "s_dirwalk: can't open %s\n", dirname);
        return;
    }
    while ((fip = readdir(dp)) != NULL) { // 如果读目录成功
        if (strcmp(fip->d_name, ".") == 0 || strcmp(fip->d_name, "..") == 0)
            continue;
        if (strlen(dirname) + strlen(fip->d_name) + 2 > sizeof(name)) {
            fprintf(stderr, "s_dirwalk: %s/%s is too long!\n", dirname, fip->d_name);
            return;
        }
        else {
            s_printf(fip->d_name, depth);
            sprintf(name, "%s/%s", dirname, fip->d_name);
            (*fcn)(name, depth);
        }
    }
    closedir(dp); // 完成后再关闭文件
}

void s_printf(char *filename, int depth) {
    while (depth > 0)
        printf(" | ");
    printf(" |-- ");
    printf("%s\n", filename);
}

```

12) 运行结果

```
[root@cvtech shiyan1]# ./tree
--tree
--cp
--cp.c
--who
--hello.c
--ls
--tree.c
--ls.c
--hello
--who.c
```

五.实验总结

1. 掌握了Linux中文件与目录的操作
2. 熟悉了把C文件生成可执行代码的方法,即gcc的运用.
3. 对 who, ls, cp, tree 等命令有了基本的认识和理解

南京工程学院

信息与通信工程学院

实 验 报 告

课 程 名 称 嵌入式系统编程

实验项目名称 进程操作

实验学生班级 信息11

实验学生姓名 余卓彦

同组学生姓名 208170809

实 验 时 间 2020.6.20

实 验 地 点 _____

实验成绩评定 _____

指导教师签字 _____

年 月 日

一. 实验目的.

- 掌握 Linux 下进程操作.
- 掌握进程概念, 熟悉进程状态及其转换, 学习进程控制相关的系统调用与函数.

二. 实验设备.

- 硬件: PC 机、嵌入式教学实验开发平台.
- 软件: Fedora Linux 操作系统、嵌入式 Linux 开发环境.

三. 实验内容: 进程状态的获取, 创建新进程, 执行新进程, 父子进程间同步, 进程间通信.

编写 parent-child、编写 execl、new-procs、编写 segfault、alarm2.

四. 实验步骤.

1. segfault.c

1) 代码:

```
int main(int argc, char **argv){  
    int a; a = *(int*) 0; }  
}
```

2) 运行结果:

```
[root@cvtech shiyan2]# ./segfault  
Segmentation fault
```

2. alarm.c

1) 代码

```
#include <stdio.h>
```

```
#include <signal.h>
```

```
#include <unistd.h>
```

```
void sig_handler(int signo){
```

```
    printf("\nB-e-e-p! 3 sencond passed.\n\n"); // 输出 (3s-次).
```

```
    alarm(3); }
```

// 时钟函数.

```
int main(void){
```

```
    if(signal(SIGALRM, sig_handler) == SIG_ERR) // 把时钟信号进行处理, 调用失败返回 SIG_ERR.
```

```
        printf("I can't catch SIGALRM\n");
```

```
    alarm(3);
```

```
    long i = 0;
```

```
    while(i != 6){
```

// 循环次数

```
        printf("I am doing hard work, i = %ld now.\n", i); // 每次循环输出 i
```

```
        sleep(1); i++; }
```

// 睡眠 1s.

```
    return 0; }
```

2) 运行结果如下:

```
[root@cvtech shiyan2]# ./alarm  
I am doing hard work. i = 0 now.  
I am doing hard work. i = 1 now.  
I am doing hard work. i = 2 now.  
B-e-e-p! 3 sencond passed.  
I am doing hard work. i = 3 now.  
I am doing hard work. i = 4 now.  
I am doing hard work. i = 5 now.  
B-e-e-p! 3 sencond passed.
```

3. exed.c

1). 代码

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int main(void) {
    printf("Here is the first line: --->1st<---\n");
    execl("/bin/date", "date", (char*)0); //执行可执行文件/bin/date.
    perror("execl() failure!\n\n"); //表示函数调用失败.
    printf("I would say good bye to you!\n");
    printf("Here is the Last line: --->LAST LINE<---\n");
    return 0;
}
```

2). 运行结果如下

```
[root@cvtech shiyan2]# ./exed
Here is the first line: ---> 1st <---
Wed Jun 24 16:24:20 CST 2020
[root@cvtech shiyan2]#
```

4. new-prcs.c

1). 代码

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int main(void) {
    pid_t pid; //定义一个进程号.
    printf("There is only one process now.\n");
    if((pid=fork()) < 0) { //如果创建子进程失败
        perror("fork error");
        exit(1);
    }
    if(pid==0) { //如果创建进程成功.
        printf("This message is printed by the child process.\n");
        execl("/bin/date", "date", (char*)0); //执行可执行文件date.
        perror("execl() failure!\n\n");
    }
    else {
        printf("PARENT PROCESS.\n");
        printf("=== The end ===\n"); return(0);
    }
}
```

2). 运行结果

```
[root@cvtech shiyan2]# ./new-prcs
There is only one process now.
This message is printed by the child process.
Wed Jun 24 16:26:37 CST 2020
PARENT PROCESS.
=== The End ===
```

5. wait.c

1) 代码

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int main(void)
{
    pid_t pid; // 定义一个进程号.
    printf("There is only one process now.\n");
    if((pid=fork()) < 0) { // 创建子进程失败.
        perror("fork error");
        exit(1);
    } // 创建子进程成功
    if(pid==0) {
        printf("This message is printed by the child process.\n");
        execl("/bin/date", "date", (char*)0); // 执行文件.
        perror("execl() failure!\n\n");
    }
    else { printf("PARENT PROCESS.\n");
        wait(0);
        printf("=== The End ===\n");
        return(0);
    }
}
```

2) 运行结果

```
[root@cvtech shiyan2]# ./wait
There is only one process now.
This message is printed by the child process.
Wed Jun 24 16:27:48 CST 2020
PARENT PROCESS.
=== The End ===
```

6. fork.c

1) 代码

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int main(void)
{
    pid_t pid; // 定义一个进程号.
    printf("--- There is only one process now. ---\n");
    sleep(5); // 睡眠5秒
    printf("--- I will going to call fork()... ---\n");
    sleep(3); // 睡眠3秒
    if((pid=fork()) < 0) { // 如果创建子进程失败.
        perror("fork error"); exit(1);
    }
    if(pid==0) { // 如果创建子进程成功.
```

```

printf("This message is printed by the child process. >>>\n");
sleep(1); //睡眠1s
printf("This message is printed by the child process. >>>\n");
sleep(1); //睡眠1s.
printf("This message is printed by the child process. >>>\n"); }
else { printf("<<< This is from PARENT PROCESS.\n");
      sleep(1);
      printf("<<< This is from PARENT PROCESS.\n");
      sleep(1);
      printf("<<< This is from PARENT PROCESS.\n"); }

sleep(8);
printf("=== I will going to the end...\n"); //输出语句
sleep(4); printf("=== The End ===\n");
return(0); }

```

```

[root@cvtech shiyan2]# ./fork
--- There is only one process now. ---
--- I will going to call fork() ... ---
This message is printed by the child process. >>>
<<< This is from PARENT PROCESS.
This message is printed by the child process. >>>
<<< This is from PARENT PROCESS.
This message is printed by the child process. >>>
<<< This is from PARENT PROCESS.
=== I will going to the end ...
=== I will going to the end ...
=== The End ===
=== The End ===

```

7. parent-child.C

1. 代码

```

int main() {
    char buf[100] = {0}; // 定义缓冲区
    pid_t cld_pid; // 定义该结构保存子进程的PID
    int fd, status; // 定义文件描述符和用于wait调用的参数
    if ((fd = open("temp", O_CREAT | O_RDWR | O_TRUNC, 0664)) == -1) { // 打开文件失败
        perror("创建文件"); exit(1); }
    strcpy(buf, "父进程数据");
    if ((cld_pid = fork()) == 0) { // 创建子进程. 如果创建成功
        strcpy(buf, "子进程数据");
        puts("子进程正在工作:");
        printf("子进程PID是%d\n", getpid()); // 输出子进程的PID.
        printf("父进程PID是%d\n", getppid()); // 输出父进程的PID.
        write(fd, buf, strlen(buf)); // 把缓冲区的内容写入fd文件里.
        close(fd); exit(0); }
    else { puts("父进程正在工作:");
        printf("父进程PID是%d\n", getpid());
        printf("子进程PID是%d\n", cld_pid);
        write(fd, buf, strlen(buf)); close(fd); }
    wait(&status); // 等待子进程结束
}

```

```
return(0); }
```

121. 运行结果

```
[root@cytech shiyan2]# ./parent-child
子进程正在工作:
子进程PID是20260
父进程正在工作:
父进程PID是20259
子进程正在工作:
子进程PID是20260
```

五. 实验总结.

1. 掌握了进程的基本概念, 熟悉了进程状态及其转换
2. 了解了进程控制相关的系统调用与函数

南京工程学院

信息与通信工程学院

实 验 报 告

课 程 名 称 嵌入式系统编程

实验项目名称 多线程与内存管理

实验学生班级 信息11

实验学生姓名 余卓豫

同组学生姓名 208170809

实 验 时 间 2020.6.20

实 验 地 点 _____

实验成绩评定 _____

指导教师签字 _____

年 月 日

一. 实验目的

- 掌握Linux下多线程与内存管理
- 掌握线程概念, 熟悉进程状态及其转换, 学习线程控制相关系统调用与函数

二. 设备

- 硬件: PC机、嵌入式教学实验开发平台
- 软件: Fedora Linux 操作系统、嵌入式Linux开发环境

三. 实验内容: 线程创建(create)、终止(exit)、线程合并(join)、线程同步(sem, mutex).

编写hello_multi、编写tworccount 1, 2、编写leak、编写lockdemo, semaphores.

四. 实验步骤

1. leak.c

1). 代码

```
#include <stdio.h>
#include <stdlib.h>
wid f(wid) {
    wid *s;
    s = malloc(50); // 分配空间
    return s;
}
int main(wid) {
    while(1) f(1); // 进行无限循环
    return 0;
}
```

2). 运行结果

```
[root@cvtech shiyan3]# ./leak
Killed
```

2. lockdemo.c

1). 代码

```
int main(int argc, char *argv[]) {
    struct flock fl = { F_WRLCK, SEEK_SET, 0, 0, 0 }; // 锁对象并初始化
    int fd; // 定义文件描述符
    fl.l_pid = getpid();
    if (argc > 1)
        fl.l_type = F_RDLCK;
    if ((fd = open("lockdemo.c", O_RDWR)) == -1) { // 打开文件如果失败
        perror("open"); exit(1);
    }
    printf("Press <RETURN> to try to get lock: ");
    getchar(); // 输入一个字符
    printf("Trying to get lock...");
    if (fcntl(fd, F_SETLKW, fl) == -1) {
        perror("fcntl"); exit(1);
    }
    printf("got lock\n");
}
```

```

printf("Press < RETURN > to release lock:");
getchar();
if (l_type == F_UNLCK;
if (fcntl(fd, F_SETLK, 0) == -1) { // 获取文件锁
    perror("fcntl"); exit(1); // 文件锁获取失败
}
printf("Unlocked.\n");
close(fd);
return 0; }

```

(2). 运行结果

```

[root@cvtech shiyan3]# ./lockdemo
Press <RETURN> to try to get lock:
Trying to get lock...got lock
Press <RETURN> to release lock:
Unlocked.

```

3. semaphores.c

(1). 代码

```

#define MAXPIDS 10
void worker(int i, void *args);
int main(int argc, char *argv[]) {
    int error, i, id, n; // 定义信号灯.
    sem_t *semlock; // 定义进程号.
    pid_t pids[MAXPIDS];
    if (argc != 2) {
        fprintf(stderr, "usage: %s numprocesses\n", argv[0]); // 输出.
        return 1; }
    n = atoi(argv[1]);
    if (n > MAXPIDS) return 1;
    #ifdef USE_SHARED_MEMORY
    if ((id = shmget(IPC_PRIVATE, sizeof(sem_t), (S_IRUSR | S_IWUSR)) == -1) {
        perror("Failed to create shared memory segment"); return 1; }
    if ((semlock = (sem_t *) shmat(id, NULL, 0)) == (void *) -1) {
        perror("Failed to attach memory segment"); return 1; }
    else { semlock = (sem_t *) malloc(sizeof(sem_t)); // 为信号灯分配内存.
    }
    #endif
    if (sem_init(semlock, 1, 1) == -1) { // 初始化一个匿名信号量.
        perror("Failed to initialize semaphore"); } // 初始化失败.
    else { for (i = 0; i < n; i++) {
        if ((pids[i] = fork()) < 0) { // 创建子进程失败.
            fprintf(stderr, "Failed to create process: %s\n", strerror(errno));
            return 1; }
            if (pids[i] == 0) { // 如果创建子进程成功
                worker(i, semlock);
                exit(0); } }
        for (i = 0; i < n; i++) wait(0); // 等待所有子进程结束
        fprintf(stderr, "workers are done\n"); }
}

```

```

#ifdef USE_SHARED_MEMORY
    if (shmdt((void *) semlock) == -1) { // 删除共享内存失败.
        perror("Failed to destroy shared memory segment");
        return 1; }
#else
    free(semlock); // 释放内存空间
#endif
return 0; }

```

(2) 运行结果:

```

[root@cvtech shiyan3]#
[root@cvtech shiyan3]# ./semaphores
Usage: ./semaphores numprocesses

```

```

4. hello-multi
#include <stdio.h>
#include <pthread.h>
#define NUM 5
main() { // 定义线程 t1, t2
    pthread_t t1, t2;
    void *print_msg(void *);
    pthread_create(&t1, NULL, print_msg, (void *) "hello"); // 创建子线程 t1.
    pthread_create(&t2, NULL, print_msg, (void *) "world\n"); // 创建子线程 t2.
    pthread_join(t1, NULL); // 等待 t1 终止
    pthread_join(t2, NULL); // 等待 t2 终止.
    void *print_msg(void *m) {
        char *cp = (char *) m;
        int i;
        for(i=0; i<NUM; i++) {
            printf("%s", cp); fflush(stdout); sleep(1); // 循环打印.
        }
        return NULL; }
}

```

(2) 运行结果

```

[root@cvtech shiyan3]# ./hello-multi
helloworld
helloworld
helloworld
helloworld
world
helloworld

```

5. incprint.c

(1) 代码.

```

#include <stdio.h>
#include <pthread.h>
#define NUM 5
int counter=0;
main() {
    pthread_t t1; // 定义线程 t1.
}

```

```

void *print_count(void *);
int i;
pthread_create(&t1, NULL, print_count, NULL); // 创建子线程t1.
for(i=0; i<NUM; i++){
    counter++;
    sleep(1); }
pthread_join(t1, NULL); // 等待t1终止.
void *print_count(void *m){
    int i;
    for(i=0; i<NUM; i++){
        printf("count=%d\n", counter); // 打印counter总数.
        sleep(1); }
    return NULL; }

```

(2). 运行结果

```

[root@cvtech shiyan3]# ./incprint
count = 0
count = 1
count = 3
count = 4
count = 5

```

6. twordcount1.C

1). 代码:

```

int total_words;
main(int ac, char *av[]) {
    pthread_t t1, t2;
    void *count_words(void *);
    if(ac!=3){
        printf("usage: %s file1 file2\n", av[0]);
        exit(1); }
    total_words=0;
    pthread_create(&t1, NULL, count_words, (void *) av[1]); // 创建子线程t1,
    pthread_create(&t2, NULL, count_words, (void *) av[2]); // 创建子线程t2.
    pthread_join(t1, NULL); // 等待t1终止.
    pthread_join(t2, NULL);
    printf("%5d: total words\n", total_words); }
void *count_words(void *f){
    char *filename = (char *) f; // 定义文件名指针.
    FILE *fp; // 定义文件描述符.
    int c, prevc = '\0';
    if((fp=fopen(filename, "r")) != NULL){ // 打开文件成功.
        while((c=getc(fp)) != EOF){
            if(!isalnum(c) && !isalnum(prevc))
                total_word++;
            prevc=c; }
        fclose(fp); } // 关闭文件.
}

```

```
else perror(filename);  
return NULL; }
```

14. 运行结果

```
[root@cvtech shiyan3]# ./twordcount1  
usage: ./twordcount1 file1 file2
```

五. 实验总结.

1. 了解了linux下多线程与内存管理机制.
2. 掌握了线程概念, 熟悉线程状态及其转换以及线程控制相关的系统调用与函数.

南京工程学院

信息与通信工程学院

实 验 报 告

课 程 名 称 嵌入式系统编程

实验项目名称 多线程与内存管理

实验学生班级 信息11

实验学生姓名 余卓豫

同组学生姓名 20810809

实 验 时 间 2020.6.20

实 验 地 点 _____

实验成绩评定 _____

指导教师签字 _____

年 月 日

一. 实验目的

- 掌握Linux下多线程内存管理
- 掌握线程概念, 熟悉进程状态及其转换, 学习线程控制相关系统调用与函数

二. 设备

- 硬件: PC机、嵌入式教学实验开发平台
- 软件: Fedora Linux操作系统、嵌入式Linux开发环境

三. 实验内容: 线程创建(create), 终止(exit), 线程合并(join), 线程同步(sem, mutex).

编写hello_multi, 编写tworcount 1, 2, 编写leak, 编写lockdemo, semaphores.

四. 实验步骤

1. leak.c

(1). 代码

```
#include <stdio.h>
#include <stdlib.h>
wid f(void) {
    wid *s;
    s = malloc(50);    // 分配空间
    return; }
int main (wid) {
    while(1) f();      // 进行无限循环
    return 0; }
```

(2). 运行结果

```
[root@cvtech shiyan3]# ./leak
Killed
```

2. lockdemo.c

(1). 代码

```
int main (int argc, char *argv[]) {
    struct flock fl = { F_WRLCK, SEEK_SET, 0, 0, 0 }; // 定义对象并初始化
    int fd;                                           // 定义文件描述符
    fl.l_pid = getpid();
    if (argc > 1)
        fl.l_type = F_RDLCK;
    if ((fd = open("lockdemo.c", O_RDWR)) == -1) { // 打开文件如果失败
        perror("open"); exit(1); }
    printf("Press <RETURN> to try to get lock: ");
    getchar();                                       // 输入一个字符
    printf("Trying to get lock...");
    if (fcntl(fd, F_SETLKW, fl) == -1) {
        perror("fcntl"); exit(1); }
    printf("got lock\n"); }
```

```

printf("Press < RETURN > to release lock:");
getchar();
if (l_type == F_UNLCK;
if (fcntl(fd, F_SETLK, 0) == -1) { // 获取文件锁
    perror("fcntl"); exit(1); // 文件锁获取失败
}
printf("Unlocked.\n");
close(fd);
return 0; }

```

(2). 运行结果

```

[root@cvtech shiyan3]# ./lockdemo
Press <RETURN> to try to get lock:
Trying to get lock...got lock
Press <RETURN> to release lock:
Unlocked.

```

3. semaphores.c

(1). 代码

```

#define MAXPIDS 10
void worker(int i, void *args);
int main(int argc, char *argv[]) {
    int error, i, id, n; // 定义信号灯.
    sem_t *semlock; // 定义进程号.
    pid_t pids[MAXPIDS];
    if (argc != 2) {
        fprintf(stderr, "usage: %s numprocesses\n", argv[0]); // 输出.
        return 1; }
    n = atoi(argv[1]);
    if (n > MAXPIDS) return 1;
    #ifdef USE_SHARED_MEMORY
    if ((id = shmget(IPC_PRIVATE, sizeof(sem_t), (S_IRUSR | S_IWUSR)) == -1) {
        perror("Failed to create shared memory segment"); return 1; }
    if ((semlock = (sem_t *) shmat(id, NULL, 0)) == (void *) -1) {
        perror("Failed to attach memory segment"); return 1; }
    else { semlock = (sem_t *) malloc(sizeof(sem_t)); // 为信号灯分配内存.
    }
    #endif
    if (sem_init(semlock, 1, 1) == -1) { // 初始化一个匿名信号量.
        perror("Failed to initialize semaphore"); // 初始化失败.
    }
    else { for (i = 0; i < n; i++) {
        if ((pids[i] = fork()) < 0) { // 创建子进程失败.
            fprintf(stderr, "Failed to create process: %s\n", strerror(errno));
            return 1; }
        if (pids[i] == 0) { // 如果创建子进程成功
            worker(i, semlock);
            exit(0); } }
        for (i = 0; i < n; i++) wait(0); // 等待所有子进程结束
        fprintf(stderr, "workers all done\n"); }
}

```

```

#ifdef USE_SHARED_MEMORY
    if (shmctl((wid *) semlock) == -1) { // 删除共享内存失败.
        perror("Failed to destroy shared memory segment");
        return 1;
    }
#else
    free(semlock); // 释放内存空间
#endif
return 0;

```

(2) 运行结果:

```

[root@cvtech shiyan3]#
[root@cvtech shiyan3]# ./semaphores
Usage: ./semaphores numprocesses

```

```

4. hello_multi
#include <stdio.h>
#include <pthread.h>
#define NUM 5
main() {
    pthread_t t1, t2; // 定义线程 t1, t2
    void *print_msg(wid *);
    pthread_create(&t1, NULL, print_msg, (wid *) "hello"); // 创建子线程 t1.
    pthread_create(&t2, NULL, print_msg, (wid *) "world\n"); // 创建子线程 t2.
    pthread_join(t1, NULL); // 等待 t1 终止
    pthread_join(t2, NULL); // 等待 t2 终止
    wid *print_msg(wid *m) {
        char *cp = (char *) m;
        int i;
        for (i = 0; i < NUM; i++) {
            printf("%s", cp); fflush(stdout); sleep(1); // 循环打印.
        }
        return NULL;
    }
}

```

(2) 运行结果

```

[root@cvtech shiyan3]# ./hello_multi
helloworld
helloworld
helloworld
world
helloworld

```

5. incprint.c

(1) 代码.

```

#include <stdio.h>
#include <pthread.h>
#define NUM 5
int counter = 0;
main() {
    pthread_t t1; // 定义线程 t1.
}

```

```

void *print_count(void *);
int i;
pthread_create(&t1, NULL, print_count, NULL); // 创建子线程t1.
for(i=0; i<NUM; i++){
    counter++;
    sleep(1); // 睡眠1s.
}
pthread_join(t1, NULL); // 等待t1终止.
void *print_count(void *m){
    int i;
    for(i=0; i<NUM; i++){
        printf("count=%d\n", counter); // 打印counter总数.
        sleep(1);
    }
    return NULL;
}

```

12. 运行结果

```

[root@cvtech shiyan3]# ./incprint
count = 0
count = 1
count = 3
count = 4
count = 5

```

6. twordcount1.c

1) 代码:

```

int total_words;
main(int ac, char *av[]) {
    pthread_t t1, t2;
    void *count_words(void *);
    if(ac!=3) {
        printf("usage: %s file1 file2\n", av[0]);
        exit(1);
    }
    total_words=0;
    pthread_create(&t1, NULL, count_words, (void *) av[1]); // 创建子线程t1.
    pthread_create(&t2, NULL, count_words, (void *) av[2]); // 创建子线程t2.
    pthread_join(t1, NULL); // 等待t1终止.
    pthread_join(t2, NULL);
    printf("%5d: total words\n", total_words);
}

void *count_words(void *f){
    char *filename = (char *) f;
    FILE *fp;
    int c, prevc = '\0';
    if((fp=fopen(filename, "r")) != NULL) {
        while((c=getc(fp)) != EOF) {
            if(!isalnum(c) && !isalnum(prevc))
                total_word++;
            prevc=c;
        }
        fclose(fp); // 关闭文件.
    }
}

```

```
else perror(filename);  
return NULL; }
```

(4). 运行结果

```
[root@cvtech shiyan3]# ./twordcount1  
usage: ./twordcount1 file1 file2
```

五. 实验总结.

1. 了解了linux下多线程与内存管理机制.
2. 掌握了线程概念, 熟悉进程状态及其转换以及线程控制相关的系统调用与函数.

南京工程学院

信息与通信工程学院

实 验 报 告

课 程 名 称 嵌入式系统编程

实验项目名称 项目管理与综合实验

实验学生班级 信息11

实验学生姓名 余卓豪

同组学生姓名 208170809

实 验 时 间 2020.6.20

实 验 地 点 _____

实验成绩评定 _____

指导教师签字 _____

年 月 日

一. 实验目的

- 掌握 Linux 下网络编程
- 掌握套接字概念, 熟悉网络编程方法, 学习网络编程相关的系统调用与函数.

二. 实验设备

- 硬件: PC 机, 嵌入式教学实验开发平台
- 软件: Fedora Linux 操作系统, 嵌入式 Linux 开发环境.

三. 实验内容

- 实验内容: 套接字创建 (socket), 绑定 (bind), 侦听 (listen), 接收 (accept).
- 编写 gethostbyname, 编写 multi-server-2, 编写 serv, cli.

四. 实验步骤

1. gethostbyname.c

(1). 代码

```
int main (int argc, char *argv[]) {
    char *hostname = "www.njit.edu.cn"; // 定义并初始化主机名.
    char ip[100];
    struct hostent *he; // 定义对象.
    struct in_addr **addr_list;
    int i;
    if ((he = gethostbyname(hostname)) == NULL) { // 如果 ip 地址为空.
        perror("gethostbyname"); return 1; // 表示解析 ip 地址失败.
    }
    addr_list = (struct in_addr **) he->h_addr_list; // 地址列表.
    for (i = 0; addr_list[i] != NULL; i++) {
        strcpy(ip, inet_ntoa(*addr_list[i])); // 把地址列表复制到 ip.
        printf("%s resolved to: %s\n", hostname, ip);
    }
    return 0; }
```

(2). 运行结果

```
[root@cvtech ~]# cd shiyan4
[root@cvtech shiyan4]# ./gethostbyname
www.njit.edu.cn resolved to : 218.2.192.10
```

2. multi-server-2

(1). 代码:

```
void *connection_handler(void *); // 与服务器端建立连接的函数.
int main (int argc, char *argv[]) {
    int socket_desc, new_socket, c, *new_sock;
    struct sockaddr_in server, client; // 定义对象地址信息.
    char *message;
    socket_desc = socket(AF_INET, SOCK_STREAM, 0); // 初始化 socket_desc.
    if (socket_desc == -1) { // socket 创建出错.
        printf("Could not create socket");
    }
    server.sin_family = AF_INET;
    server.sin_addr.s_addr = INADDR_ANY;
    server.sin_port = htons(8888);
```

```

if (bind(socket_desc, (struct sockaddr *)&server, sizeof(server)) < 0) { // IP地址绑定
    puts("bind failed"); // 端口号出错.
    return 1; }
puts("bind done");
listen(socket_desc, 3); // 监听套接字 socket_desc.
puts("Waiting for incoming connections...");
c = sizeof(struct sockaddr_in);
while ((new_socket = accept(socket_desc, (struct sockaddr *)&client, (socklen_t *)&c)) > 0) {
    puts("Connection accepted");
    message = "Hello client, I have received your connection. And now I will
        assign a handler for you\n";
    write(new_socket, message, strlen(message)); // 把 message 写入 new_socket 里
    pthread_t sniffer_thread; // 启动线程.
    new_sock = malloc(1); // 对 new_socket 分配内存空间.
    *new_sock = new_socket;
    if (pthread_create(&sniffer_thread, NULL, connection_handler, (void *) new_sock) < 0) {
        perror("could not create thread"); // 创建子线程失败.
        return 1; }
    puts("Handler assigned"); }
if (new_socket < 0) {
    perror("accept failed");
    return 1; }
return 0;
}

void *connection_handler(void *socket_desc) { // 声明函数 connection_handler.
    // 定义套接字描述符.
    int sock = *(int *) socket_desc;
    int read_size;
    char *message, client_message[2000];
    message = "Greetings! I am your connection handler\n";
    write(sock, message, strlen(message)); // 把 message 写入 sock 里.
    message = "Now type something and I shall repeat what you type\n";
    write(sock, message, strlen(message));
    while ((read_size = recv(sock, client_message, 2000, 0)) > 0) {
        write(sock, client_message, strlen(client_message)); }
    if (read_size == 0) {
        puts("Client disconnected"); // 如果未连接上.
        fflush(stdout); }
    else if (read_size == -1) {
        perror("recv failed"); }
    free(socket_desc); // 释放套接字内存空间.
    return 0; }

```

12. 运行结果

```

[root@cvtech shiyan4]# ./multi-server-2
bind done
Waiting for incoming connections...

```

4. cli.c.

11. 代码

```
int main(int argc, char *argv[]) {
    int socket_desc, fd;
    struct sockaddr_in server;
    char message[1024];
    socket_desc = socket(AF_INET, SOCK_STREAM, 0);
    if (socket_desc == -1) {
        printf("Could not create socket");
        // 定义套接字, 文件描述符.
        // 定义服务器套接字地址.
        // 如果创建套接字失败.
    }
    server.sin_addr.s_addr = inet_addr(argv[1]);
    server.sin_family = AF_INET;
    server.sin_port = htons(8765);
    if (connect(socket_desc, (struct sockaddr *)&server, sizeof(server)) < 0) {
        puts("connect error");
        // 连接服务器地址.
        // 连接失败.
        return 1;
    }
    puts("Connected\n");
    if ((fd = open("/dev/input/mice", O_RDONLY)) == -1) {
        perror("cannot open /dev/input/mice");
        // 打开文件mice失败.
        return 1;
    }
    while (read(fd, message, 1024) > 0) {
        if (send(socket_desc, message, strlen(message), 0) < 0) {
            puts("Send failed");
            // 把message写入文件.
            // 发送文件.
            return 1;
        }
    }
    return 0;
}
```

12. 运行结果

```
[root@cvtech ~]# cd shiyan4
[root@cvtech shiyan4]# ./cli
Segmentation fault
```

5. bma.c

11. 代码

```
int main(int argc, char *argv[]) {
    int socket_desc;
    struct sockaddr_in server;
    socket_desc = socket(AF_INET, SOCK_STREAM, 0);
    if (socket_desc == -1) {
        printf("Could not create socket");
        // 定义套接字.
        // 定义服务器套接字地址.
        // 初始化.
        // 创建失败.
    }
    server.sin_family = AF_INET;
    server.sin_addr.s_addr = INADDR_ANY;
    server.sin_port = htons(8888);
    if (bind(socket_desc, (struct sockaddr *)&server, sizeof(server)) < 0) {
        puts("bind failed");
        // 绑定端口失败.
        return 1;
    }
    puts("bind done");
    return 0;
}
```

12) 运行结果

```
[root@cvtech shiyan4]# ./bind
bind done
[root@cvtech shiyan4]#
```

6. accept.c

1) 代码

```
#include <stdio.h>
#include <sys/socket.h>
#include <arpa/inet.h>

int main(int argc, char *argv[]) {
    int socket_desc, new_socket, c; // 定义套接字描述符
    struct sockaddr_in server, client; // 定义服务器套接字地址
    socket_desc = socket(AF_INET, SOCK_STREAM, 0); // 初始化
    if (socket_desc == -1) { // 创建失败
        printf("Could not create socket");
    }
    server.sin_family = AF_INET;
    server.sin_addr.s_addr = INADDR_ANY;
    server.sin_port = htons(8888);
    if (bind(socket_desc, (struct sockaddr *)&server, sizeof(server)) < 0) { // 绑定端口失败
        puts("bind failed");
        return 1;
    }
    puts("bind done"); // 完成IP地址绑定端口号
    listen(socket_desc, 3); // 将socket_desc转换为监听套接字
    puts("waiting for incoming connections...");
    c = sizeof(struct sockaddr_in); // 把大小赋值
    new_socket = accept(socket_desc, (struct sockaddr *)&client, (socklen_t *)&c); // 接收一个
    if (new_socket < 0) { // 接收失败
        perror("accept failed");
        return 2;
    }
    puts("connection accepted");
    return 0;
}
```

1) 运行结果

```
[root@cvtech shiyan4]# ./accept
bind done
waiting for incoming connections...
```

五. 实验总结

1. 掌握了Linux下网络编程
2. 掌握了套接字概念, 熟悉网络编程方法, 了解了网络编程相关的系统调用与函数。